

Introducción

En este módulo estudiaremos la tarea denominada etiquetado morfosintático (en inglés *part-of-speech tagging* o *POS-tagging*). Esta tarea consiste en asignar a cada palabra de un texto una categoría gramatical y otra información adicional (como pueden ser varias subcategorías, el lema asociado, etc.) Esta es una tarea fundamental en el procesamiento del lenguaje natural, aunque no está exenta de problemas que no están todavía totalmente resueltos. El lenguaje natural es ambiguo desde muchos puntos de vista, y también lo es en el morfosintático. Una determinada forma (como por ejemplo *casa*) puede tener varias interpretaciones morfosintáticas, puede ser un sustantivo común femenino singular (con lema *casa*) y también una forma de presente o de imperativo del verbo *casar*. Los etiquetadores morfosintáticos deberán proporcionar la interpretación adecuada según el contexto de utilización; por lo tanto deberán desambiguar las diferentes posibilidades.

Antes de abordar el etiquetado morfosintático veremos también el análisis morfológico, es decir, el análisis que nos permite determinar el lema y la categoría gramatical (y otras subcategorizaciones) de una determinada forma. Una vez seamos capaces de hacer el análisis morfológico, pasaremos a estudiar diversas técnicas que nos permitirán etiquetar textos desde el punto de vista morfosintático y que intentarán desambiguar (con mayor o menor éxito) las diferentes posibilidades.

De esta unidad dispones de los siguientes archivos:

- Estos mismos apuntes en PDF: [05-Etiquetado_morfosintático-spa.pdf](#)
- Los programas y archivos necesarios: [programas5-spa.zip](#)
- El archivo Notebook de Jupyter: [cap5-spa.ipynb](#)
- El enlace al notebook en Google Colaboratory: <https://colab.research.google.com/github/aoliverg/python/blob/master/notebooks/cap5-spa.ipynb>

5.1. Morfología computacional

5.1.1. El formalismo de descomposición morfológica

Existen toda una serie de formalismos para describir la morfología de una lengua. En este capítulo sólo presentaremos uno, que es fácil de implementar y nos servirá para comprender los mecanismos fundamentales: el formalismo de descomposición morfológica ([Alshawi, 1992](#)).

La idea básica de este formalismo es sencilla y se basa en dos tipos de conocimiento:

- Un diccionario que contiene información morfosintáctica base o la palabra que se considera forma de referencia.
- Reglas que contienen información sobre la morfología de la lengua.

En el programa-5-1.py podemos ver una primera implementación muy sencilla de esta idea:

```
raiz="cant"
t1="o"
t2="as"
t3="a"
t4="amos"
t5="áis"
t6="an"

print(raiz+t1)
print(raiz+t2)
```

```
print(raiz+t3)
print(raiz+t4)
print(raiz+t5)
print(raiz+t6)
```

Si ejecutamos este programa, obtenemos la siguiente salida:

```
canto
cantas
canta
cantamos
cantáis
cantan
```

En este programa definimos una serie de variables: una que contiene el lema de un verbo y otros que contienen las terminaciones de presente de indicativo. Mediante el operador + que concatena cadenas obtenemos las formas correspondientes al presente de indicativo.

Siguiendo esta idea básica crearemos un formalismo para expresar las reglas (que además de las formas queremos que nos den una etiqueta de categoría gramatical y subcategorizaciones). También utilizaremos un formalismo para expresar el diccionario de lemas, que además del propio lema nos indicará el tipo de flexión que sigue (es decir, el paradigma flexivo).

Como formalismo para las reglas proponemos una serie de valores separados por dos puntos (:):

```
terminación_forma:terminación_lema:etiqueta:paradigma
```

Por ejemplo:

```
o:ar:VMIP1S:V1
as:ar:VMIP2S:V1
a:ar:VMIP3S:V1
amos:ar:VMIP1P:V1
áis:ar:VMIP2P:V1
an:ar:VMIP3P:V1
```

Para las etiquetas utilizamos las etiquetas [EAGLES para el castellano](#). En el archivo reglas.txt podemos observar un conjunto más extenso de reglas morfológicas.

Para el formalismo para el diccionario de lemas seguiremos una idea similar:

```
lema: paradigma
```

Que en el ejemplo sería:

```
cantar: V1
```

En el archivo diccionario.txt podemos observar un diccionario más completo.

Ahora necesitamos un programa que nos permita generar todas las formas a partir de las reglas y el diccionario (programa-5-2.py)

```
freglas=open("reglas.txt","r")
reglas=[]
while True:
    linea=freglas.readline().rstrip()
    if not linea:break
    reglas.append(linea)
freglas.close()

fdiccionario=open("diccionario.txt","r")
while True:
    linea=fdiccionario.readline().rstrip()
    if not linea:break
    (lema,tipo)=linea.split(":")
    for regla in reglas:
        (tf,tl,etiqueta,tipo2)=regla.split(":")
        if ((tipo2 == tipo)&(lema.endswith(tl))):
            print(lema[0 : (len(lema)-len(tl))]+tf,lema,etiqueta)
fdiccionario.close()
```

que nos proporciona como salida:

```
canto cantar VMIP1S
cantas cantar VMIP2S
canta cantar VMIP3S
cantamos cantar VMIP1P
cantáis cantar VMIP2P
cantan cantar VMIP3P
como comer VMIP1S
comes comer VMIP2S
come comer VMIP3S
comemos comer VMIP1P
coméis comer VMIP2P
comen comer VMIP3P
vivo vivir VMIP1S
vives vivir VMIP2S
vive vivir VMIP3S
vivimos vivir VMIP1P
vivís vivir VMIP2P
...
```

Estamos hablando de análisis morfológico, pero en realidad lo que hemos hecho es un programa que genera formas con su lema y una etiqueta morfosintáctica, lo que llamamos diccionario morfológico. Los programas de análisis morfológico a menudo funcionan con un diccionario morfológico.

En el siguiente programa (programa-5-3.py) cargamos un gran diccionario morfológico del castellano obtenido del analizador Freeling (diccionario-freeling-spa.txt) y llevamos a cabo el análisis de las palabras que indica el usuario (el programa finaliza cuando el usuario introduce un espacio en blanco).

```
import codecs
diccionario={}
archivo_diccionario=codecs.open("diccionario-freeling-spa.txt","r",encoding="utf-8")

for entrada in archivo_diccionario:
    entrada=entrada.rstrip()
    campos=entrada.split(":")
    forma=campos[0]
    lema=campos[1]
    etiqueta=campos[2]
    diccionario[forma]=diccionario.get(forma,"")+ "\t"+lema+"\t"+etiqueta
print("DICCIONARIO CARGADO")
while 1:
    palabra=input("Introduce la palabra a analizar: ")
    if palabra==" ":
        break
    if palabra in diccionario:
        print("ANALISIS:",palabra,diccionario[palabra])
    else:
        print("PALABRA DESCONOCIDA")
```

que ofrece, por ejemplo, la siguiente salida:

```
Introduce la palabra a analizar: perro
ANALISIS: perro perro NCMS000
```

y si la palabra es desconocida:

```
Introduce la palabra a analizar: crocodrilo
PALABRA DESCONOCIDA
```

5.2. Análisis morfológico

En este apartado vamos a construir un analizador morfológico, es decir, un programa que es capaz de darnos los análisis morfológicos de todas las palabras de un texto. Para las palabras ambiguas desde el punto de vista morfosintático, nos devolverá todas las posibles interpretaciones. Con lo que hemos hecho hasta ahora tenemos todos los componentes:

- Un programa que cargue el diccionario morfológico del español (el programa-5-3.py)
- Un programa capaz de leer un documento y segmentarlo en oraciones y tokenizarlo (por ejemplo, el programa-4-3.py que vimos en la unidad anterior)
- En el programa-5-4.py podemos observar una primera versión del programa (que analizará el archivo noticia.txt, que contiene un fragmento de noticia). Utiliza también el archivo spanish.pickle que contiene el segmentador).

```
import codecs
import nltk
```

```
from nltk.corpus.reader.plaintext import PlaintextCorpusReader
from nltk.tokenize import RegexpTokenizer

diccionario={}
archivo_diccionario=codecs.open("diccionario-freeling-spa.txt","r",encoding="utf-8")

for entrada in archivo_diccionario:
    entrada=entrada.rstrip()
    camps=entrada.split(":")
    if len(camps)>=3:
        forma=camps[0]
        lema=camps[1]
        etiqueta=camps[2]
        if forma in diccionario:
            diccionario[forma]=diccionario.get(forma,"")+ " "+lema+" "+etiqueta
        else:
            diccionario[forma]=lema+" "+etiqueta

segmentador= nltk.data.load("spanish.pickle")
tokenizador=RegexpTokenizer("\w+|[\^\w\s]+")

corpus = PlaintextCorpusReader(".", 'noticia.txt',word_tokenizer=tokenizador,sent_tokenizer=segmentador)

for forma in corpus.words():
    if forma in diccionario:
        info=diccionario[forma]
    else:
        info="DESCONOCIDA"
    print(forma+" "+info)

que ens proporciona la sortida (mostrem únicament un fragmento):

Putin DESCONOCIDA
apenas apenas RG apenas CS apenar VMIP2S0
condena condena NCFS000 condenar VMIP3S0 condenar VMM02S0
la la NCMS000 el DA0FS0 lo PP3FSA0
decisión decisión NCFS000
de de NCFS000 de SP
Trump DESCONOCIDA
de de NCFS000 de SP
```

' DESCONOCIDA
ceder ceder VMN0000
' DESCONOCIDA
el el DA0MS0
Golán DESCONOCIDA
a a NCFS000 a SP
Israel DESCONOCIDA
Ha DESCONOCIDA
sido ser VSP00SM
una uno DI0FS0 uno PI0FS0 unir VMM03S0 unir VMSP1S0 unir VMSP3S0
comedida comedir VMP00SF
reacción reacción NCFS000
crítica crítico AQ0FS00 crítica NCFS000 crítico NCFS000
, DESCONOCIDA

Como vemos, hay algunos problemas, como por ejemplo:

- Las palabras que están en la primera posición de una oración, y por lo tanto están escritas en mayúsculas, a pesar de estar en el diccionario, las etiqueta como desconocidas. Para solucionar esto, primero buscaremos el diccionario las palabras tal y como aparecen en el texto, si no las encuentra, entonces las buscará pasada a minúscula, y si aún así no la encuentra, la marcará como desconocida.
- Los signos de puntuación los marca como desconocidos, ya que son tokens que no aparecen en el diccionario morfológico. La solución es sencilla y consiste en incluir los signos de puntuación en el diccionario morfológico o bien ponerlas en el propio programa.

Veamos la nueva implementación en el programa-5-5.py

```
import codecs
import nltk
from nltk.corpus.reader.plaintext import PlaintextCorpusReader
from nltk.tokenize import RegexpTokenizer

diccionario={}
archivo_diccionario=codecs.open("diccionario-freeling-spa.txt","r",encoding="utf-8")

for entrada in archivo_diccionario:
    entrada=entrada.rstrip()
    camps=entrada.split(":")
    if len(camps)>=3:
        forma=camps[0]
        lema=camps[1]
        etiqueta=camps[2]
        if forma in diccionario:
```

```
diccionario[forma]=diccionario.get(forma,"")+ " "+lema+" "+etiqueta
else:
    diccionario[forma]=lema+" "+etiqueta

#Añadimos los signos de puntuación
diccionario[""]="" Fe'
diccionario[""]="" Fe"
diccionario['.']=. Fp'
diccionario[',']=, Fc'
diccionario[';']=; Fx'
diccionario[':']=: Fd'
diccionario['(']='( Fpa'
diccionario[')']=) Fpt'
diccionario['[']=[ Fca'
diccionario['']']=] Fct'

segmentador= nltk.data.load("spanish.pickle")
tokenizador=RegexpTokenizer("\w+|[\^\w\s]+")

corpus = PlaintextCorpusReader(".", 'noticia.txt',word_tokenizer=tokenizador,sent_tokenizer=segmentador)

for forma in corpus.words():
    if forma in diccionario:
        info=diccionario[forma]
    elif forma.lower() in diccionario:
        info=diccionario[forma.lower()]
    else:
        info="DESCONOCIDA"
    print(forma+" "+info)

Que ofrece como salida:

Putin DESCONOCIDA
apenas apenas RG apenas CS apenas VMIP2S0
condena condena NCFS000 condenar VMIP3S0 condenar VMM02S0
la la NCMS000 el DA0FS0 lo PP3FSA0
decisión decisión NCFS000
de de NCFS000 de SP
Trump DESCONOCIDA
de de NCFS000 de SP
```

' ' Fe
ceder ceder VMN0000
' ' Fe
el el DA0MS0
Golán DESCONOCIDA
a a NCFS000 a SP
Israel DESCONOCIDA
Ha ha I haber VAIP3S0 haber VMIP3S0
sido ser VSP00SM
una uno DI0FS0 uno PI0FS0 unir VMM03S0 unir VMSP1S0 unir VMSP3S0
comedida comedir VMP00SF
reacción reacción NCFS000
crítica crítico AQ0FS00 crítica NCFS000 crítico NCFS000
, , Fc

Aún quedan problemas por resolver, como los nombres propios, algún aspecto mal tokenizado, etc. También pueden aparecer palabras desconocidas que son correctas, pero que no estén recogidas en el diccionario morfológico, etc. En próximos apartados de este mismo capítulo iremos presentando soluciones a estas cuestiones.

5.3. Etiquetado morfosintático

5.3.1. Introducción

En este módulo estudiaremos la tarea llamada etiquetado morfosintático (en inglés *part-of-speech tagging* o *POS-tagging*). Esta tarea consiste en asignar a cada palabra de un texto una categoría gramatical y otra información adicional (como pueden ser varias subcategorías, el lema asociado, etc.) Esta es una tarea fundamental en el procesamiento del lenguaje natural, aunque no está exenta de problemas que no están todavía totalmente resueltos.

El lenguaje natural es ambiguo desde muchos puntos de vista, y también lo es en el morfosintático. Una determinada forma (como por ejemplo casa) puede tener varias interpretaciones morfosintáticas, puede ser un sustantivo común femenino singular (con lema casa) y también una forma de presente o de imperativo del verbo casar. Los etiquetadores morfosintáticos deberán intentar dar la interpretación adecuada según el contexto donde aparece una palabra; por lo tanto deberán desambiguar las diferentes posibilidades.

En el módulo veremos varias técnicas que nos permitirán etiquetar textos desde el punto de vista morfosintático y que intentarán desambiguar (con mayor o menor éxito) las diferentes posibilidades.

5.3.2. Etiquetado morfosintático vs. análisis morfológico

En el apartado anterior hemos estudiado la función denominada análisis morfológico, que consiste en asignar a cada palabra de un texto todas las posibles análisis morfológicos. A continuación podemos ver el análisis morfológico, llevado a cabo con el analizador Freeling, de la oración:

Mañana por la mañana lloverá.

Vemos que la palabra mañana puede ser tanto un adverbio como un sustantivo. El análisis morfosintático nos ofrece todas las interpretaciones posibles:

Mañana	por	la	mañana	lloverá	.
mañana <i>RG</i> 0.538032	por <i>SP</i> 1	el <i>DA0FS0</i> 0.98926	mañana <i>RG</i> 0.538408	llover <i>VMIF3S0</i> 1	. <i>Fp</i> 1
mañana <i>NCCS000</i> 0.46127		lo <i>PP3FSA0</i> 0.010734	mañana <i>NCCS000</i> 0.461592		
mañana <i>NP00000</i> 0.000697837		la <i>NCMS000</i> 6.20105e-06			

Un analizador morfosintático ofrece la misma salida, pero desambiguada, es decir, elige una de las posibilidades de cada palabra. Vemos ahora el análisis morfosintático hecha por Freeling de la misma oración:

Mañana	por	la	mañana	lloverá	.
mañana <i>RG</i>	por <i>SP</i>	el <i>DA0FS0</i>	mañana <i>NCCS000</i>	llover <i>VMIF3S0</i>	. <i>Fp</i>

El etiquetado morfosintático es una tarea básica para muchas tareas de procesamiento del lenguaje natural. Si queremos hacer un análisis sintáctico de una oración, un paso previo es conocer la categoría gramatical de cada palabra. Disponer de textos etiquetados a escala morfosintáctica es interesante para muchos estudios. Podemos saber cuáles son los sustantivos más utilizados en un corpus, ver todas las apariciones de un verbo independientemente de la forma concreta, etc. El etiquetado morfosintático también se utiliza para extraer los términos más relevantes de un determinado documento o conjunto de documentos. Estas técnicas también se utiliza para la clasificación de documentos y recuperación de información.

5.3.4. Etiquetador para el inglés

NLTK proporciona un etiquetador para el inglés que funciona bastante bien y que se puede usar fácilmente de manera directa, como en el programa-5-6.py.

```
import nltk

oracion="They refuse to permit us to obtain the refuse permit"

palabras = nltk.tokenize.word_tokenize(oracion)

analisis=nltk.pos_tag(palabras)

print(analisis)
```

que ofrece la siguiente salida:

5. Etiquetado morfosintático

[('They', 'PRP'), ('refuse', 'VBP'), ('to', 'TO'), ('permit', 'VB'), ('us', 'PRP'), ('to', 'TO'), ('obtain', 'VB'), ('the', 'DT'), ('refuse', 'NN'), ('permit', 'NN')]

Hay que tener en cuenta que la palabra *permit* la ha etiquetado correctamente como verbo y como sustantivo. Las etiquetas utilizadas son las propias de un etiquetario (tagset), o conjunto de etiquetas, determinado. En este caso concreto se utilizan el tagset WSJ (Wall Street Journal), que es el siguiente:

Tag - Function

CC - coordinating conjunction

CD - cardinal number

DT - determiner

EX - existential ``there''

FW - foreign word

IN - preposition

JJ - adjective

JJR - adjective, comparative

JJS - adjective, superlative

MD - modal

NN - non-plural common noun

NNP - non-plural proper noun

NNPS - plural proper noun

NNS - plural common noun

of - the word ``of''

PDT - pre-determiner

POS - possessive

PRP - pronoun

puncf - final punctuation (period, question mark and exclamation mark)

punc - other punctuation

RB - adverb

RBR - adverb, comparative

RBS - adverb, superlative

RP - particle

TO - the word ``to''

UH - interjection

VB - verb, base form

VBD - verb, past tense

VBG - verb, gerund or present participle

VBN - verb, past participle

VBP - verb, non-3rd person

VBZ - verb, 3rd person

WDT - wh-determiner

5. Etiquetado morfosintático

WP - wh-pronoun
WRB - wh-adverb
sym - symbol
2 - ambiguously labelled

Como veremos cuando empezamos a construir etiquetadores para otras lenguas, como el español, las etiquetas que utilizaremos serán diferentes. Hay una propuesta de etiquetario universal (*universal tagset*). A continuación podemos observar este etiquetario.

Tag Meaning - English Examples

ADJ - adjective - new, good, high, special, big, local
ADP - adposition - on, of, at, with, by, into, under
ADV - adverb - really, already, still, early, now
CONJ - conjunction - and, or, but, if, while, although
DET - determiner, article - the, a, some, most, every, no, which
NOUN - noun - year, home, costs, time, Africa
NUM - numeral - twenty-four, fourth, 1991, 14:24
PRT - particle - at, on, out, over per, that, up, with
PRON - pronoun - he, their, her, its, my, I, us
VERB - verb - is, say, told, given, playing, would
. - punctuation marks- . , ; !
X - other - ersatz, esprit, dunno, gr8, univeristy

NLTK nos permite convertir las etiquetas de un determinado etiquetario a las del etiquetario universal. Lo podemos ver en el programa-5-6b.py.

```
import nltk

oracion="They refuse to permit us to obtain the refuse permit"
palabras = nltk.tokenize.word_tokenize(oracion)
analisis=nltk.pos_tag(palabras)

for ana in analisis:
    forma=ana[0]
    etiqueta=ana[1]
    universal=nltk.tag.mapping.map_tag('en-ptb', 'universal', etiqueta)
    print(forma,etiqueta,universal)
```

que nos proporciona la siguiente salida:

```
They PRP PRON
refuse VBP VERB
to TO PRT
permit VB VERB
```

us PRP PRON
to TO PRT
obtain VB VERB
the DT DET
refuse NN NOUN
permit NN NOUN

5.4. Entrenamiento de etiquetadores

5.4.1. El etiquetador por defecto

En este apartado se presenta un etiquetador muy simple, que lo único que hace es etiquetar todas las palabras con una etiqueta determinada, es decir, etiqueta todas las palabras con la misma etiqueta. Para determinar qué etiqueta elegiremos lo que haremos primero será calcular cuál es la etiqueta más frecuente. Para lograr esto haremos uso de corpus ya etiquetados: para el inglés utilizaremos el Brown Corpus y para el español el CESS_ESP.

Para el inglés:

En un intérprete interactivo para calcular la etiqueta más frecuente haremos:

```
>>> import nltk
>>> from nltk.corpus import brown
>>> tags=[tag for (word,tag) in brown.tagged_words()]
>>> nltk.FreqDist(tags).max()
'NN'
```

Y para definir un etiquetador por defecto hacemos:

```
>>> from nltk.tokenize import word_tokenize
>>> from nltk.tokenize import word_tokenize
>>> oracio="they refuse to permit us to obtain the refuse permit"
>>> tokens=word_tokenize(oracio)
>>> default_tagger=nltk.DefaultTagger('NN')
>>> default_tagger.tag(tokens)
[('they', 'NN'), ('refuse', 'NN'), ('to', 'NN'), ('permit', 'NN'), ('us', 'NN'), ('to', 'NN'), ('obtain', 'NN'), ('the', 'NN'), ('refuse', 'NN'), ('permit', 'NN')]
```

Como podemos observar, etiqueta todas las palabras con "NN".

Para el español:

```
>>> import nltk
>>> from nltk.corpus import cess_esp
>>> tags=[tag for (word,tag) in cess_esp.tagged_words()]
```

```
>>> nltk.FreqDist(tags).max()
'sps00'
```

Nos puede sorprender que la etiqueta más frecuente en español sea la correspondiente a la preposición ya que esperaríamos que fuera también la correspondiente a sustantivo. Lo que pasa es que en el etiquetario español los sustantivos tienen diversas posiciones para la categoría y varias subcategorizaciones (género y número). Si modificamos las dos últimas líneas para hacer que el program mire sólo la primera posición de la etiqueta, obtendremos que la categoría más frecuente es 'n' (sustantivo).

```
>>> tags=[tag[0] for (word,tag) in cess_esp.tagged_words()]
>>> nltk.FreqDist(tags).max()
'n'
```

Podríamos hacer también un etiquetador por defecto del español indicando que la etiqueta por defecto fuese 'n'. Pero realmente preferiríamos indicar la etiqueta completa más frecuente para los sustantivos.

```
>>> tags=[tag for (word,tag) in cess_esp.tagged_words()]
>>> nltk.FreqDist(tags).most_common(10)
[('sps00', 25272), ('ncms000', 11428), ('Fc', 11420), ('ncfs000', 11008), ('da0fs0', 6838), ('da0ms0', 6012), ('rg', 5937), ('Fp', 5866), ('cc', 5854), ('ncmp000', 5711)]
```

Que nos da que los nombres comunes masculinos singulares son ligeramente más frecuentes que los nombres comunes femeninos singulares. Ahora podemos definir un etiquetador por defecto para el español:

```
>>> from nltk import word_tokenize
>>> oracion="Mañana por la mañana lloverá."
>>> tokens=word_tokenize(oracion)
>>> default_tagger=nltk.DefaultTagger('ncms000')
>>> default_tagger.tag(tokens)
[('Mañana', 'ncms000'), ('por', 'ncms000'), ('la', 'ncms000'), ('mañana', 'ncms000'), ('lloverá', 'ncms000'), ('.', 'ncms000')]
```

Ya nos podemos imaginar que este etiquetador no funcionará demasiado bien. En el siguiente apartado aprenderemos a evaluar etiquetadores y podremos ver la precisión de este etiquetador. También puede servir para los casos que tengamos una palabra desconocida, ya que le podremos asignar la etiqueta más frecuente (sin contar la de preposición, ya que siendo una categoría cerrada es prácticamente imposible que sea desconocida).

5.4.2. El etiquetador por unigramas

El etiquetador por defecto estudiado en el apartado anterior etiqueta todas las palabras con la etiqueta más frecuente en todo el corpus. En este apartado y los próximos estudiaremos una serie de etiquetadores llamados genéricamente etiquetadores por n-gramas. Un n-grama es una combinación de n elementos. En general estos etiquetadores aprenden a partir del corpus teniendo en cuenta un contexto de n palabras. En el caso del etiquetador por unigramas lo único que tenemos en cuenta es la propia palabra a etiquetar, sin ningún contexto. El etiquetador por unigramas etiquetará cada palabra con la etiqueta más frecuente para esa palabra.

Con NLTK es muy sencillo crear un etiquetador por unigramas. Primero lo haremos para el inglés y luego para el español.

Para el inglés

```
>>> import nltk
>>> from nltk.corpus import brown
>>> tagged_sents=brown.tagged_sents()
>>> unigram_tagger=nltk.UnigramTagger(tagged_sents)
>>> oracio="they refuse to permit us to obtain the refuse permit"
>>> tokens=nltk.word_tokenize(oracio)
>>> unigram_tagger.tag(tokens)

[('they', 'PPSS'), ('refuse', 'VB'), ('to', 'TO'), ('permit', 'VB'), ('us', 'PPO'), ('to', 'TO'), ('obtain', 'VB'), ('the', 'AT'), ('refuse', 'VB'), ('permit', 'VB')]
```

Si nos fijamos en el resultado, vemos que *refuse* la etiqueta las dos veces que aparece como verbo, ya que esta es la etiqueta más frecuente para esta palabra.

Para el español

Haremos lo mismo, pero esta vez en un programa (programa-5-7.py):

```
import nltk
from nltk.corpus import cess_esp
from nltk.tokenize import word_tokenize
tagged_sents=cess_esp.tagged_sents()
unigram_tagger=nltk.UnigramTagger(tagged_sents)
oracio="Mañana por la mañana lloverá."
tokens=word_tokenize(oracio)
analisis=unigram_tagger.tag(tokens)
print(analisis)
```

que proporciona la salida:

```
[('Mañana', 'rg'), ('por', 'sps00'), ('la', 'da0fs0'), ('mañana', 'rg'), ('lloverá', None), ('.', 'Fp')]
```

Para entrenar etiquetadores también podemos utilizar nuestros propios corpus etiquetados. En el siguiente ejemplo utilizaremos un fragmento del Wikicorpus del español. Si nos fijamos en el formato de este corpus etiquetado veremos que es forma, lema, etiqueta y probabilidad de la etiqueta separados por tabulador. Tendremos que crear un código capaz de leer este corpus y crear las `tagged_sents` como listas de `tagged_words` donde cada `tagged_word` es una tupla forma, etiqueta. Lo podemos hacer con el programa-5-8.py (en los archivos que se distribuyen el archivo que contiene el fragmento del Wikicorpus está comprimido en zip, no olvides descomprimirlo antes de ejecutar el programa):

```
import codecs
import nltk

entrada=codecs.open("fragmento-wikicorpus-tagged-spa.txt", "r", encoding="utf-8")
```

```
tagged_words=[]
tagged_sents=[]
for linia in entrada:
    linia=linia.rstrip()
    if linia.startswith("<") or len(linia)==0:
        if len(tagged_words)>0:
            tagged_sents.append(tagged_words)
            tagged_words=[]
        else:
            camps=linia.split(" ")
            forma=camps[0]
            lema=camps[1]
            etiqueta=camps[2]
            tupla=(forma,etiqueta)
            tagged_words.append(tupla)

unigram_tagger=nlk.UnigramTagger(tagged_sents)
oracio="mañana por la mañana lloverá"
tokens=nlk.tokenize.word_tokenize(oracio)
analisi=unigram_tagger.tag(tokens)
print(analisi)

y proporciona la siguiente salida:

[('mañana', 'NCCS000'), ('por', 'SP'), ('la', 'DA0FS0'), ('mañana', 'NCCS000'), ('lloverá', None)]
```

5.4.3. El etiquetador por bigramas

El etiquetador por unigramas no tiene en cuenta el contexto de aparición de las palabras, y las etiqueta siempre con la etiqueta más frecuente para esa palabra. En este apartado vamos a construir un etiquetador por bigramas que tiene en cuenta la propia palabra a etiquetar y la palabra anterior:

Para el inglés

```
>>> import nlk
>>> from nlk.corpus import brown
>>> tagged_sents=brown.tagged_sents()
>>> bigram_tagger=nlk.BigramTagger(tagged_sents)
>>> oracio="they refuse to permit us to obtain the refuse permit"
>>> tokens=word_tokenize(oracio)
>>> tokens=nlk.tokenize.word_tokenize(oracio)
```

```
>>> bigram_tagger.tag(tokens)
>>> bigram_tagger.tag(tokens)
[('they', 'PPSS'), ('refuse', 'VB'), ('to', 'TO'), ('permit', 'VB'), ('us', 'PPO'), ('to', 'TO'), ('obtain', 'VB'), ('the', 'AT'), ('refuse', 'NN'), ('permit', 'NN')]
```

Fijémonos que ahora puede etiquetar el segundo *refuse* como nombre, ya que tiene en cuenta el contexto inmediato.

Para el español

Modificamos el programa-5-8.py para obtener el programa-5-9.py:

```
import codecs
import nltk

entrada=codecs.open("fragmento-wikicorpus-tagged-spa.txt","r",encoding="utf-8")

tagged_words=[]
tagged_sents=[]
for linia in entrada:
    linia=linia.rstrip()
    if linia.startswith("<") or len(linia)==0:
        if len(tagged_words)>0:
            tagged_sents.append(tagged_words)
            tagged_words=[]
        else:
            camps=linia.split(" ")
            forma=camps[0]
            lema=camps[1]
            etiqueta=camps[2]
            tupla=(forma,etiqueta)
            tagged_words.append(tupla)

bigram_tagger=nltk.BigramTagger(tagged_sents)
oracio="mañana por la mañana lloverá"
tokens=nltk.tokenize.word_tokenize(oracio)
analisi=bigram_tagger.tag(tokens)
print(analisi)
```

Que ofrece la siguiente salida:

```
[('mañana', None), ('por', None), ('la', None), ('mañana', None), ('lloverá', None)]
```


Es decir, que no ha podido etiquetar ninguna palabra. Esto se debe a un fenómeno conocido como dispersión de datos. Hay muchos más unigramas que bigramas en un corpus. Si entrenamos un etiquetador por bigramas, necesitaremos un corpus más grande para poder encontrar suficientes evidencias de cada bigrama de la oración a analizar, como no siempre es posible disponer de grandes corpus, se puede recurrir a la técnica conocida como *backoff*. A esta técnica también se la conoce como combinación de etiquetadores. En el programa-5-10.py combinamos un etiquetador por bigramas con uno por unigramas.

```
import codecs
import nltk

entrada=codecs.open("fragmento-wikicorpus-tagged-spa.txt","r",encoding="utf-8")

tagged_words=[]
tagged_sents=[]
for linea in entrada:
    linea=linea.rstrip()
    if linea.startswith("<") or len(linea)==0:
        if len(tagged_words)>0:
            tagged_sents.append(tagged_words)
            tagged_words=[]
        else:
            camps=linea.split(" ")
            forma=camps[0]
            lema=camps[1]
            etiqueta=camps[2]
            tupla=(forma,etiqueta)
            tagged_words.append(tupla)

unigram_tagger=nltk.UnigramTagger(tagged_sents)
bigram_tagger=nltk.BigramTagger(tagged_sents,backoff=unigram_tagger)
oracio="mañana por la mañana lloverá"
tokens=nltk.tokenize.word_tokenize(oracio)
analisi=bigram_tagger.tag(tokens)
print(analisi)

y nos ofrece la siguiente salida:

[('mañana', 'NCCS000'), ('por', 'SP'), ('la', 'DA0FS0'), ('mañana', 'NCCS000'), ('lloverá', None)]
```

y como vemos, ha podido etiquetar la mayoría las palabras, ya que con el etiquetador por bigramas no ha podido, pero sí utilizando el de unigramas.

5.4.5. El etiquetador por trigramas

El etiquetador por trigramas etiqueta una palabra teniendo en cuenta el contexto formado por las dos palabras anteriores. En este caso, el problema de la dispersión de datos será aún más pronunciado. En el programa-5-11.py implementamos un etiquetador por trigramas que se combina con uno por bigramas y a su vez por uno por unigramas.

```
import codecs
import nltk

entrada=codecs.open("fragmento-wikicorpus-tagged-spa.txt","r",encoding="utf-8")

tagged_words=[]
tagged_sents=[]
for linia in entrada:
    linia=linia.rstrip()
    if linia.startswith("<") or len(linia)==0:
        if len(tagged_words)>0:
            tagged_sents.append(tagged_words)
            tagged_words=[]
        else:
            camps=linia.split(" ")
            forma=camps[0]
            lema=camps[1]
            etiqueta=camps[2]
            tupla=(forma,etiqueta)
            tagged_words.append(tupla)

unigram_tagger=nltk.UnigramTagger(tagged_sents)
bigram_tagger=nltk.BigramTagger(tagged_sents,backoff=unigram_tagger)
trigram_tagger=nltk.TrigramTagger(tagged_sents,backoff=bigram_tagger)
oracio="mañana por la mañana lloverá"
tokens=nltk.tokenize.word_tokenize(oracio)
analisi=bigram_tagger.tag(tokens)
print(analisi)

que proporciona la siguiente análisis:

[('mañana', 'NCCS000'), ('por', 'SP'), ('la', 'DA0FS0'), ('mañana', 'NCCS000'), ('lloverá', None)]
```

5.4.5. Tratamiento de palabras desconocidas: diccionario morfológico, etiquetador por afijos y etiquetador por defecto

En los programas anteriores hemos visto que un etiquetador entrenado con un corpus no será capaz de etiquetar palabras que no aparezcan en el corpus. Para solucionar esto utilizaremos 3 estrategias:

5. Etiquetado morfosintático

- entrenar un etiquetador por unigramas a partir de un diccionario morfológico, que en este caso será el del propio Freeing y que se puede descargar junto con los programas de este capítulo.
- entrenar un etiquetador por afijos que lo que hace es utilizar todas las palabras del diccionario morfológico (y en su defecto podrían utilizarse las propias palabras del corpus), para aprender las etiquetas más frecuentes según las terminaciones de las palabras.
- y si todo esto falla, usar un etiquetador por defecto, que use la etiqueta más frecuente en nuestro corpus de aprendizaje

Vemos primero estos tres entrenamientos por separado y luego lo combinaremos todo en un único etiquetador.

Etiquetador por unigramas a partir de un diccionario morfológico

Utilizaremos esta parte de código (programa-5-12.py):

```
import codecs
import nltk

entrada=codecs.open("diccionario-freeling-spa.txt","r",encoding="utf-8")

tagged_words=[]
tagged_sents=[]
cont=0
for linia in entrada:
    cont+=1
    if cont==10000:
        break
    linia=linia.rstrip()
    camps=linia.split(":")
    forma=camps[0]
    lema=camps[1]
    etiqueta=camps[2]
    tupla=(forma,etiqueta)
    tagged_words.append(tupla)
tagged_sents.append(tagged_words)

unigram_tagger_diccionari=nltk.UnigramTagger(tagged_sents)
```

Fíjate que contamos las líneas y cuando llega a la 10000 paramos el entrenamiento para que no tarde demasiado. Para hacer las pruebas será suficiente y en el momento de hacer el entrenamiento definitivo eliminaremos (comentaremos) estas líneas de código de manera que haga el entrenamiento con todo el diccionario. Este programa no ofrece ninguna salida.

Entrenamiento de un etiquetador por afijos utilizando el diccionario morfológico

Podemos encontrar la implementación en el programa-5-13.py que es exactamente igual que el anterior pero cambia la última línea y ahora es:

```
affix_tagger=nltk.AffixTagger(tagged_sents, affix_length=-3, min_stem_length=2)1
```

que permite entrenar el etiquetador por afijos teniendo en cuenta los afijos con tres caracteres siempre y cuando la raíz que quede tenga 2 o más caracteres. Este programa tampoco devuelve ninguna salida.

Determinación de la etiqueta más frecuente y entrenamiento del etiquetador por defecto

El programa-5-14.py nos devuelve las 10 etiquetas más frecuentes:

```
import codecs
import nltk

entrada=codecs.open("fragmento-wikicorpus-tagged-spa.txt","r",encoding="utf-8")

tagged_words=[]
tagged_sents=[]
for linia in entrada:
    linia=linia.rstrip()
    if linia.startswith("<") or len(linia)==0:
        #nueva linia
        if len(tagged_words)>0:
            tagged_sents.append(tagged_words)
            tagged_words=[]
        else:
            camps=linia.split(" ")
            forma=camps[0]
            lema=camps[1]
            etiqueta=camps[2]
            tupla=(forma,etiqueta)
            tagged_words.append(tupla)
tags=[]
for ts in tagged_sents:
    for wt in ts:
        tags.append(wt[1])

mft=nltk.FreqDist(tags).most_common(10)
print("Etiqueta más frecuente: ",mft)

default_tagger=nltk.DefaultTagger("NP00000")

Que ofrece como salida:

Etiquetas más frecuente: [('SP', 574004), ('NP00000', 330331), ('NCMS000', 251790), ('NCFS000', 247815), ('Fc', 212993), ('Fp', 170279), ('DA0MS0', 170139), ('DA0FS0', 139776), ('CC', 122292), ('NCMP000', 107563)]
```

Y volvemos a obtener que la más frecuente (de las categorías abiertas) es el sustantivo, concretamente el nombre propio. Ahora podemos crear el etiquetador por defecto utilizando esta etiqueta

Pondremos todo esto junto en el siguiente apartado, y además, aprenderemos a almacenar etiquetadores.

5.5. Almacenamiento de etiquetadores

En el apartado anterior hemos aprendido a entrenar y combinar etiquetadores. Cada vez que queríamos etiquetar una oración, entrenábamos un etiquetador y luego etiquetábamos. Como el entrenamiento es lento, nos interesa entrenar una vez y poder guardar el etiquetador ya entrenado de manera que lo podamos utilizar tantas veces como queramos.

En el programa-5-15.py entrenamos y almacenamos un etiquetador. Fíjate en todos los etiquetadores diferentes que entrenamos, como los combinamos, y como finalmente almacenamos el último, que de hecho está combinado con todos los demás. Fíjate también como usamos el módulo pickle. Y también ten en cuenta que hemos comentado las líneas que limitan el número de entradas del diccionario morfológico que utiliza para entrenar. Si ves que tarda mucho en entrenar, vuelve a descomentar estas líneas:

```
import codecs

import nltk

import pickle

entrada=codecs.open("fragmento-wikicorpus-tagged-spa.txt","r",encoding="utf-8")

tagged_words=[]
tagged_sents=[]
tagged_sents_per_unigrams=[]
cont=0
for linia in entrada:
    #cont+=1
    #if cont==10000:
    #    break
    linia=linia.rstrip()
    if linia.startswith("<") or len(linia)==0:
        #nova linia
        if len(tagged_words)>0:
            tagged_sents.append(tagged_words)
            tagged_sents_per_unigrams.append(tagged_words)
            tagged_words=[]
        else:
            camps=linia.split(" ")
            forma=camps[0]
            lema=camps[1]
```

```
etiqueta=camps[2]
tupla=(forma,etiqueta)
tagged_words.append(tupla)

if len(tagged_words)>0:
    tagged_sents.append(tagged_words)
    tagged_sents_per_unigrams.append(tagged_words)
    tagged_words=[]

diccionario=codecs.open("diccionario-freeling-spa.txt","r",encoding="utf-8")

cont=0
for linia in diccionario:
    #cont+=1
    #if cont==10000:
    #    break
    linia=linia.rstrip()
    camps=linia.split(":")
    if len(camps)>=3:
        forma=camps[0]
        lema=camps[1]
        etiqueta=camps[2]
        tupla=(forma,etiqueta)
        tagged_words.append(tupla)
    tagged_sents_per_unigrams.append(tagged_words)

default_tagger=nlTK.DefaultTagger("NP00000")
affix_tagger=nlTK.AffixTagger(tagged_sents_per_unigrams, affix_length=-3,
min_stem_length=2,backoff=default_tagger)
unigram_tagger_diccionario=nlTK.UnigramTagger(tagged_sents_per_unigrams,backoff=affix_tagger)
unigram_tagger=nlTK.UnigramTagger(tagged_sents,backoff=unigram_tagger_diccionario)
bigram_tagger=nlTK.BigramTagger(tagged_sents,backoff=unigram_tagger)
trigram_tagger=nlTK.TrigramTagger(tagged_sents,backoff=bigram_tagger)

sortida=open('etiquetador-spa.pkl', 'wb')
pickle.dump(trigram_tagger, sortida, -1)
sortida.close()
```

Lo que es importante tener en cuenta en el programa anterior es que estamos proporcionando un corpus y un diccionario. El corpus nos proporciona información de forma y etiqueta dentro del contexto de la oración. En cambio, el diccionario sólo nos da información sobre formas y sus etiquetas de una manera totalmente fuera de

contexto, ya que la palabra que aparece en el diccionario antes de otra sólo guarda una relación alfabética. Por este motivo, en todos los entrenamientos que supongan un contexto (bigrama y trigrama) sólo podemos utilizar la información que proviene del corpus. En cambio, para los entrenamientos que no se tenga en cuenta el contexto (afijos y unigramas) podemos utilizar tanto la información que aparece en el corpus como la que aparece en el diccionario. Ten en cuenta que en el programa utilizamos dos listas para almacenar la información que utilizamos para entrenar:

- `tagged_sents_per_unigrams`: donde ponemos la información del corpus y del diccionario.
- `tagged_sents`: donde solo ponemos la información del corpus.

Las líneas:

```
for linia in entrada:
```

```
    #cont+=1
```

```
    #if cont==10000:
```

```
        # break
```

```
y
```

```
cont=0
```

```
for linia in diccionario:
```

```
    #cont+=1
```

```
    #if cont==10000:
```

```
        # break
```

sirven para limitar la información que se carga o bien del corpus o bien del diccionario, o de ambos. Como el programa tarda mucho en ejecutarse, puedes descomentar (quitar el símbolo "#") de delante de las líneas.

En los archivos de esta unidad encontrarás el etiquetador entrenado (`etiquetador-spa.pkl`) resultante para que lo podáis utilizar en el siguiente programa sin esperar a que se complete el entrenamiento.

Ahora en el archivo `etiquetador-spa.pkl` tenemos un etiquetador que podemos cargar siempre que queramos de manera muy rápida. Fijate en el programa-5-16.py:

```
import nltk
```

```
import pickle
```

```
import nltk
```

```
entrada=open('etiquetador-spa.pkl','rb')
```

```
etiquetador=pickle.load(entrada)
```

```
entrada.close()
```

```
oracio="Mañana por la mañana lloverá."
```

```
tokenizador=nltk.tokenize.RegexpTokenizer('\w+|^\w\s|+')
```

```
tokens=tokenizador.tokenize(oracio)
```

```
analisis=etiquetador.tag(tokens)
```

```
print(analisis)
```

Que ofrece el análisis:

```
[('Mañana', 'NCCS000'), ('por', 'SP'), ('la', 'DA0FS0'), ('mañana', 'NCCS000'), ('lloverá', 'VMIF3S0'), ('.', 'Fp')]
```

5.6. Evaluación de etiquetadores

En los apartados anteriores hemos aprendido a crear etiquetadores estadísticos. También hemos aprendido a usarlos para etiquetar textos e intuitivamente hemos visto que funcionan bastante bien, aunque pueden etiquetar mal algunas palabras. Cuando desarrollamos un etiquetador, nos interesaría saber qué precisión logramos. NLTK nos proporciona una manera muy sencilla de evaluar etiquetadores. Empezamos evaluando un etiquetador para el inglés (programa-5-17.py)

```
import nltk

from nltk.corpus import brown

brown_tagged_sents=brown.tagged_sents()

print("TOTAL ORACIONES:", len(brown_tagged_sents))

train_sents=brown_tagged_sents[:10000]

test_sents=brown_tagged_sents[56001:]

default_tagger=nltk.DefaultTagger("NN")

affix_tagger=nltk.AffixTagger(train_sents, affix_length=-3, min_stem_length=2)

unigram_tagger=nltk.UnigramTagger(train_sents, backoff=affix_tagger)

bigram_tagger=nltk.BigramTagger(train_sents, backoff=unigram_tagger)

trigram_tagger=nltk.TrigramTagger(train_sents, backoff=bigram_tagger)

precisio=trigram_tagger.evaluate(test_sents)

print("PRECISION: ",precisio)
```

El programa primero nos indicará el número total de oraciones del corpus. Fíjate que crea un conjunto de oraciones de entrenamiento (train_sents) con las primeras 10000 oraciones del corpus; y uno de test con las oraciones finales, de la 56001 hasta la final. Después de crear el etiquetador con el método evaluate evalúa la precisión utilizando las oraciones de test. Lo que hace el programa es etiquetar estas oraciones y compararlas con las etiquetas reales. Si ejecutas el programa, obtendrás la siguiente salida:

```
TOTAL ORACIONES: 57340
PRECISION: 0.8931815568586869
```

Cambia ahora el número de oraciones para entrenar el etiquetador y pon el máximo (sin coger oraciones de test), es decir, 56000. Si

ejecutas ahora el programa, la precisión pasa a ser de:

```
PRECISION: 0.9220420834770611
```

Vemos que cuanto mayor sea el corpus de entrenamiento, obtendremos una mejor precisión.

Vamos a evaluar el etiquetador del español que hemos entrenado y almacenado en el apartado anterior. Lo hacemos en el programa-5-18.py

```
import nltk
import pickle
```



```
import codecs

#cargamos el etiquetador
entrada=open('etiquetador-spa.pkl','rb')
etiquetador=pickle.load(entrada)
entrada.close()

#cargamos las oraciones del corpus de test

entrada=codecs.open("fragmento-wikicorpus-tagged-spa.txt","r",encoding="utf-8")

tagged_words=[]
test_tagged_sents=[]
for linea in entrada:
    linea=linea.rstrip()
    if linea.startswith("<") or len(linea)==0:
        #nueva linea
        if len(tagged_words)>0:
            test_tagged_sents.append(tagged_words)
            tagged_words=[]
    else:
        camps=linea.split(" ")
        forma=camps[0]
        lema=camps[1]
        etiqueta=camps[2]
        tupla=(forma,etiqueta)
        tagged_words.append(tupla)

precisio=etiquetador.evaluate(test_tagged_sents)
print("PRECISION: ",precisio)
```

Si nos fijamos, utilizamos un fragmento nuevo de wikicorpus ya etiquetado (fragmento-wikicorpus-test-tagged-spa.txt). Primero cargamos el etiquetador almacenado y luego cargamos el nuevo corpus añadiéndolo al test_tagged_sents, que son las que utilizaremos para evaluar con el método evaluate. Este programa nos proporciona la siguiente salida:

```
PRECISION: 0.9938872748117409
```

Recordemos que el corpus que estamos usando es un corpus etiquetado automáticamente con Freeling, y por tanto, el 99.3% de precisión que obtenemos no es real. Deberíamos utilizar corpus etiquetados manualmente, o bien etiquetados automáticamente y revisados.